

Trust, Augment, or Replace? What a Cross-Project Defect Predictor Is Worth on 1,267 Real Defects from Defects4J and BugsInPy

Vijay Prasad Javvadi¹

¹Independent Researcher, Plainsboro, New Jersey, USA

Corresponding author: Vijay Prasad Javvadi, 9007 Tamarron Drive, Plainsboro, NJ 08536, USA

vijay@vijayjavvadiresearch.ai ORCID 0009-0004-1192-6906

Abstract

A team that adopts a file-level defect predictor trained on other projects faces three questions before it can act on the tool's output. Can the probabilities be trusted on a project the model has never seen? Should the team add its own failure-time signals to improve them? And would a large language model do the job as well from the same information? We answer all three on 1,267 real defects from Defects4J (774 bugs, 16 Java projects) and BugsInPy (493 bugs, 17 Python projects), 6,716 candidate files in total, under a leave-one-project-out protocol in which every prediction is made for a project excluded from training. Each defect is presented as a failure event whose candidates are the files changed by the fix together with sibling source files from the same directories at the pre-fix revision, so that the ranking task is between files that look alike. Trust: the predictor ranks a fix file first in 54% of events (against 24% for a flat prior) and its probabilities are well calibrated up to 0.6 (expected calibration error 0.025 pooled, 0.023 on Java and 0.065 on Python); above 0.6 it is over-confident, so a threshold of 0.7 selects files that are defective 56% of the time, not 70%. Augment: the failure-time signals available for these corpora are constant within an event, so a rule that adds them cannot change the ranking and only moves the probabilities off calibration (error 0.091), and a gradient-boosted fusion model re-fitted on the same inputs is worse on both axes (Precision@1 down 4.5 points, error 0.082). Replace: given exactly the predictor's features, Claude Sonnet 4.6 matches the predictor's pooled ranking (Precision@1 0.532 against 0.541, difference inside the bootstrap interval) but the parity is concentrated on three Defects4J projects, Lang, Math and Time, where it ranks the fix file first in 90% of events against the predictor's 51%; on the other 30 projects the predictor leads by 7.6 points, and GPT-4o-mini trails everywhere. Both models are two to three times worse calibrated than the predictor. The practical rule is to use the predictor's raw probability for ordering and for thresholds up to about 0.6, not to wrap it in fusion logic, and to treat an LLM's success on the best-known benchmark projects as evidence of recall rather than of attribution. The corpus, harness, prompts, model outputs and every number are archived on Zenodo.

Keywords: defect prediction; cross-project prediction; probability calibration; fault localization; test-failure triage; Defects4J; BugsInPy; large language models; benchmark contamination

1 Introduction

A continuous-integration test has just failed. The developer has a short list of files that could be responsible, typically the files in the directories the change touched, and has to decide where to look first. Many teams now have a defect predictor available for this decision, and increasingly it is one trained on other projects, because a new project has no defect history of its own. The predictor returns a probability per file. Before that number can drive anything, whether a ticket assignment, a merge gate or simply the order in which a developer reads files, the team needs to know three things that the tool's documentation does not say.

The first is whether the probability means what it says on a project the model has never seen. A predictor that ranks the right file first but reports 0.9 on attributions that are correct half the time is unusable for any workflow that acts on the number rather than on the rank (Guo et al., 2017; Niculescu-Mizil and Caruana, 2005). This is the property the fault-localization literature does not report: recent LLM-based and spectrum-based localizers (Yang et al., 2024; Kang et al., 2024; Rezaalipour and Furia, 2024) publish ranking accuracy only, and the defect-prediction studies that do report calibration (Shahini et al., 2024; Hong et al., 2024) do so at commit granularity and rarely on a project excluded from training.

The second is whether the team should improve the predictor by adding what it knows at failure time: which kinds of files the change touched, whether the exception is unusual, whether a file is a known flake. It is natural to assume that more signal helps. Whether it does on real defects, and whether the signal is even available in a form a per-file ranker can use, has not been measured.

The third is whether the predictor is worth keeping at all now that a language model can be asked the same question. If a frontier model given the same information ranks and calibrates as well, the training pipeline is unnecessary. The comparison has a complication that this paper takes seriously: the two corpora are among the most heavily used benchmarks in the program-repair literature (Xia et al., 2023), and recent work finds that large models have memorised parts of them (Ramos et al., 2024). A model that recognises a bug from its project name and file paths is not attributing, it is recalling.

We answer the three questions as three research questions, trust (RQ1), augment (RQ2) and replace (RQ3), on 1,267 real defects from Defects4J (Just et al., 2014) and BugsInPy (Widyasari et al., 2020) across 33 projects, under a leave-one-project-out protocol: train on 32 projects, score the 33rd, so that every number in the paper is an estimate for an unseen project. The methods compared are the cross-project predictor a team would adopt, a flat prior that stands for having no tool, the two natural ways of augmenting the predictor (a hand-written rule and a gradient-boosted fusion model), and two prompted frontier models, Claude Sonnet 4.6 and GPT-4o-mini, shown exactly the information the fusion model sees.

RQ1 (trust). How well does the cross-project predictor rank, and how well calibrated are its file-level probabilities, when every prediction is made on a project excluded from training?

RQ2 (augment). Does adding the failure-time signals available for these corpora, through a rule or a learned fusion model, improve ranking or calibration?

RQ3 (replace). Does a prompted frontier LLM, given the same per-event and per-file information as the fusion model, match the trained predictor on ranking and calibration, and does its performance depend on how well known the project is?

The answers are a qualified yes, no, and not from attribution. The predictor ranks a fix file first in 54% of events against 24% for the flat prior, and its probabilities track the observed defect rate closely up to 0.6, with an expected calibration error of 0.025; above 0.6 it is overconfident, and we say where (Section 3.1). The failure-time signals these corpora can supply without executing tests turn out to be constant within an event, so the rule fusion reproduces the predictor’s ranking exactly while tripling its calibration error, and the learned fusion loses 4.5 points of Precision@1 (Section 3.2). Claude Sonnet 4.6 matches the predictor’s pooled ranking, but the match comes from three Defects4J projects (Lang, Math, Time) on which it ranks the fix file first in 90% of events; on the remaining 1,082 events the predictor leads by 7.6 points with a confidence interval that excludes zero, and GPT-4o-mini shows no such concentration (Section 3.3).

The paper makes four contributions. It gives a team an answer it can act on: a file-level predictor trained on other projects can be used for ordering and, up to a probability of about

0.6, for thresholds, on a project it has never seen, and we report the per-language split rather than a pooled number alone. It shows, from the data rather than by assumption, that the failure-time signals two standard corpora can supply are event-level constants, so that fusion on top of a calibrated score has only a downside, and it reports what that downside is. It compares the predictor with two frontier LLMs under strict information parity, to our knowledge the first such comparison at file granularity, and shows that the stronger model’s apparent parity is a property of the best-known benchmark projects rather than of the task. And it releases a corrected corpus of 1,267 events and 6,716 candidate files whose features are computed at the original pre-fix revision for both languages, with the harness, the four methods, the LLM prompts and raw responses, and every figure, archived on Zenodo and mirrored on GitHub.

Section 1.1 places the work against fault localization, calibrated defect prediction and bug attribution. Section 2 defines the methods, the protocol, the metrics, the data sources and the corpus construction. Section 3 reports the three answers, a sensitivity analysis and per-project statistics. Section 4 interprets them, states the practical rule and lists the threats. Section 5 concludes and states the limitations.

1.1 Related work

The evaluation reported here sits at the intersection of three literatures that have accelerated over the 2024–2026 window: LLM-based fault localization, calibrated just-in-time defect prediction, and refinements to the SZZ family of bug-attribution algorithms. We situate the paper’s contribution against each, and we anchor the empirical protocol in the real-defect-corpus literature that Defects4J (Just et al., 2014) and BugsInPy (Widyasari et al., 2020) have established.

1.2 Real-defect corpora and their standard use

Defects4J (Just et al., 2014) provides a curated database of reproducible real bugs across mature open-source Java projects with author-verified fix commits and failing test cases; v2.0.1 covers 17 projects. BugsInPy (Widyasari et al., 2020) is its Python analogue, providing real bugs across 17 open-source Python projects, each with a reproducible failing test and an author-verified fix commit. Both have become de facto benchmarks for fault localization and automated program repair (*e.g.*, Yang et al., 2024; Kang et al., 2024; Rezaalipour and Furia, 2024; Xia et al., 2023). Their common usage pattern in fault localization is to score suspiciousness at the line or statement level using coverage or LLM prompting; the present paper instead uses them at the file level as the ground truth for a defect-attribution task in which the candidate set is derived from the fix commit’s modified files and their siblings at the pre-fix revision.

Several recent studies build directly on these corpora. LLMAO (Yang et al., 2024) evaluates an adapter-based LLM localizer at line granularity on both corpora. Kang, An and Yoo (Kang et al., 2024) run GPT-3.5 and GPT-4 on Defects4J at method and line granularity. Rezaalipour and Furia (Rezaalipour and Furia, 2024) benchmark seven localizers on 135 BugsInPy faults. All three inherit the classical fault-localization framing: rank statements or methods by a suspiciousness score and evaluate on held-out bugs from the same corpus without an explicit cross-project generalisation guarantee, and all three report ranking accuracy only. Our contribution is orthogonal on three axes. We make calibration a first-class outcome and report expected calibration error alongside every ranking metric. We adopt a strict leave-one-project-out protocol so that every headline number is a cross-project estimate. And we work at file granularity on a candidate set of look-alike files, the operational unit of triage rather than the diagnostic unit of localization. The use of Defects4J in LLM evaluation has a further consequence that this paper measures directly: Ramos et al. (Ramos et al., 2024) report that large models reproduce Defects4J bug content at rates consistent with memorisation, and the program-repair literature

has evaluated on the Defects4J v1.2 projects, Chart, Closure, Lang, Math, Mockito and Time, since its beginning (Xia et al., 2023). We therefore report LLM results with and without those projects.

1.3 LLM-based fault localization

Yang et al. (Yang et al., 2024) presented LLMAO at ICSE 2024, the first LLM-based approach that localizes faulty lines without test-coverage information, evaluated on Defects4J and BugsInPy at the line level. Kang, An and Yoo (Kang et al., 2024) evaluated GPT-3.5 and GPT-4 on Defects4J and reported that prompt-only localization is competitive with spectrum-based baselines when the model has a bug report but degrades on larger codebases. Rezaalipour and Furia (Rezaalipour and Furia, 2024) confirmed that Ochiai (Abreu et al., 2009) remains competitive on real Python faults. Earlier learning-to-rank approaches such as DeepFL (Li et al., 2019) and FLUCCS (Sohn and Yoo, 2017) combine spectrum-based features with code and change metrics at method or statement granularity and require execution spectra. Rather than compare line-level localizers, we frame the problem at the file level on a candidate set a CI system could expose, measure the calibration of the emitted probabilities under leave-one-project-out, and ask whether fusing a pre-execution prior with the failure-time features these corpora supply buys anything. We also prompt Claude Sonnet 4.6 and GPT-4o-mini as file-level attribution methods restricted to exactly the features the fused model observes (Section 2.2), which yields a like-for-like ranking-and-calibration comparison between a trained cross-project prior and frontier LLMs.

1.4 Just-in-time defect prediction and probability calibration

Just-in-time defect prediction remains the most active sub-area of defect prediction, and calibration has moved from a marginal to a central concern. Zhou, Han and Lo (Zhou et al., 2025) proposed a fusion framework combining hand-crafted expert features with deep commit representations; its motivation, that simple and complex models are complementary, is the design principle behind our learned fusion, and our result is a caution about applying it where the extra features carry no within-event information. Shahini, Metzger and Pohl (Shahini et al., 2024) applied conformal prediction to JIT-SDP; conformal prediction and post-hoc calibration are complementary to what we measure, which is the calibration of raw classifier outputs without either. Li et al. (Li et al., 2024) showed that the choice of sampling technique materially changes downstream calibration, which is relevant to a protocol that changes the training distribution per fold. Chouchen et al. (Chouchen et al., 2026) argued that simple gradient-boosting variants match deep networks on file-level defect prediction; our predictor is such a model. Hong et al. (Hong et al., 2024) documented practitioner scepticism of poorly calibrated risk scores at Atlassian, the operational failure mode RQ1 addresses. The process-metric features we use descend from Nagappan and Ball (Nagappan and Ball, 2005), Moser et al. (Moser et al., 2008), Kamei et al. (Kamei et al., 2013) and Rahman and Devanbu (Rahman and Devanbu, 2013); the cross-project concern follows Zimmermann et al. (Zimmermann et al., 2009) and Tantithamthavorn et al. (Tantithamthavorn et al., 2017). Calibration methodology is drawn from Niculescu-Mizil and Caruana (Niculescu-Mizil and Caruana, 2005) and Guo et al. (Guo et al., 2017).

1.5 SZZ and bug-attribution algorithms

The SZZ family (Śliwerski et al., 2005; Bird et al., 2009; Herzig et al., 2013; Rosa et al., 2021) remains the standard for linking a bug-fix commit back to its bug-introducing change. Rani, Petrulio and Bacchelli (Rani et al., 2024) refined SZZ by mining Mozilla bug-report discussions,

and Perez-Rosero et al. (Perez-Rosero et al., 2025) introduced WIA-SZZ, which groups bug-inducing commits with their fixes into work items. Both presuppose that a fix commit exists. In the live-triage scenario that motivates this paper no fix exists yet: a test has just failed and the developer needs to know which of a handful of files to inspect first. Post-execution attribution is the complement to SZZ, operating on the incoming failure rather than on the eventual fix. Herzig, Just and Zeller’s misclassification analysis (Herzig et al., 2013) applies to both regimes and informs our threats.

1.6 Test-failure triage and flakiness

Parry et al. (Parry et al., 2025) analysed 810 flaky-test instances across 24 Java projects and showed that most flakiness manifests as co-occurring cluster failures with a shared root cause. Their finding motivates the per-file flake-rate input of the two fusion designs; as Section 2.6 reports, that input is not recoverable from Defects4J or BugsInPy without re-executing tests, which is one reason the fusion arms have nothing to work with here. Broader work on CI cost (Vasilescu et al., 2015) provides the operational context in which the trust question matters.

2 Materials & Methods

We define the four internal attribution methods (Section 2.1), the two LLM baselines (Section 2.2), the leave-one-project-out protocol (Section 2.3), the metrics (Section 2.4), the data sources (Section 2.5), the corpus-construction steps (Section 2.6), the computing infrastructure (Section 2.7) and the use of generative AI in the study (Section 2.8).

2.1 Four attribution methods

Each method takes as input a failure event e consisting of a candidate set of files F_e : the non-test source files modified in the ground-truth fix commit (the real defects for that event) together with distractor files from the same directories at the pre-fix revision (Section 2.6). Each candidate file carries seven process-metric features computed from the project’s git history at the pre-fix revision: `commit_count`, `unique_developers`, `lines_added`, `lines_deleted`, `code_churn` (added plus deleted), `file_age_days` (first to last commit touching the file) and `commit_frequency` (commits per week of age). Each event also carries nine failure-side columns, described under the rule fusion below. Each method emits a score $s(f, e) \in [0, 1]$ for every file $f \in F_e$; higher scores are ranked first, and tied scores are broken by a random permutation drawn once with a fixed seed (42), never by file path or corpus order, because the corpus lists fix files before distractors.

Cross-project predictor (pre_only). The predictor a team would adopt. Each candidate file is scored by a gradient-boosting classifier (scikit-learn, 100 estimators, maximum depth 3, fixed random state) trained on the seven process-metric features of the other 32 projects, after standardisation fitted on the training fold. No failure-side column is used and no prior-defect feature is used, because a per-file defect history would have to come from the same corpus that supplies the labels.

Flat prior (fail_only). The no-tool baseline. Every candidate file is scored 1.0, so the ranking is the random tie-break; this stands for inspecting the candidates in arbitrary order.

Rule fusion (fused_rule). The first way a team would augment the predictor: a hand-written additive rule on its score. The nine failure-side columns are an exception-type index, an HTTP status, a locator-heal flag, a historical flake rate, and five change-topology flags (`app_code_changed`, `validator_changed`, `schema_changed`, `fixture_changed`, `config_changed`). With s_{pre} the predictor’s score,

$$s_{\text{rule}} = \text{clip}(s_{\text{pre}} + 0.08 \cdot \text{app_code_changed} + 0.05 \cdot \text{validator_changed} + 0.03 \cdot \text{schema_changed} - 0.05 \cdot \text{fixture_changed} - \text{flake_pen}, 0, 1),$$

where $\text{flake_pen} = \max(0, \text{flake_rate} - 0.10) \cdot 0.5$. The coefficients were fixed before the real-data evaluation (they come from an earlier study of ours, T8 in Section 4.6) and were not re-tuned. In these corpora the exception type, HTTP status, locator-heal flag and flake rate each take a single value across the whole corpus, because recovering them would require re-executing every failing test, and the five change-topology flags are derived from the fix-file paths and are therefore identical for every candidate of an event (Section 2.6). The rule consequently adds the same offset to every file of an event: it cannot change the within-event ranking, and its only effect is on the probability scale. We keep it in the comparison because that is precisely the effect a team wrapping a calibrated score in a rule would see.

Learned fusion (fused_ml). The second way a team would augment the predictor: a gradient-boosting classifier (150 estimators, maximum depth 3, fixed random state) trained on the seven process-metric features, the nine failure-side columns and the predictor’s own score (17 inputs), with the same binary label. Hyperparameters were fixed in advance; no per-fold retuning and no post-hoc recalibration (isotonic or otherwise) is applied to any method, so every reported probability is the classifier’s raw output on the held-out project. Because the nine failure-side columns are constant or event-level constants, the learned fusion tests one thing: what it costs to re-fit a higher-capacity model on top of a calibrated score when no new within-event signal is available.

2.2 LLM attribution baselines

We evaluate two prompted LLMs as fifth and sixth attribution methods: Claude Sonnet 4.6 (Anthropic, accessed through the Anthropic Messages API, October 2026) and GPT-4o-mini (OpenAI, Chat Completions API, October 2026). Each of the 1,267 events is presented as a single prompt at temperature 0, and the model must return strict JSON containing a complete ranking of the event’s candidate files together with a per-file confidence in $[0, 1]$. The raw confidence is the row score and enters the ECE computation as-is; the model’s explicit ranking breaks ties among equal confidences. Candidates are shown in a deterministic per-event shuffle (seeded from the SHA-256 of the event identifier), never in corpus order. Responses failing strict-JSON parsing trigger one retry; a second failure would fall back to the presented order with uniform confidence and be recorded as a parse failure. In the runs reported here this fallback was never triggered. The Claude run consumed 1,142,698 input and 274,943 output tokens; The Claude run cost US\$7.55; the GPT-4o-mini run incurred no charge on the author’s account. All outputs are scored by the same metric implementation as the four internal methods.

The design constraint is strict information parity with the learned fusion: the prompt contains the project name and language, the event-level failure columns (exception type and index, HTTP status, flake rate, and which change-topology flags are set), and for each candidate its path and the seven process-metric values, and nothing else. The LLM sees no labels, no file contents, no diffs, no test source and no fix commits. Because the LLMs are not trained on corpus data by us, every LLM prediction is an out-of-project prediction by construction. The comparison therefore isolates one question: given the same observation, does a frontier LLM’s

in-context reasoning recover what a trained cross-project prior extracts from those features? The prompt does, however, reveal the project name and the file paths, which is enough for a model that has seen the benchmark during pre-training to recognise a bug; Section 3.3 tests for that directly.

2.3 Leave-One-Project-Out protocol

The cross-project predictor and the learned fusion are trained models, and the rule fusion depends on the predictor’s score. To eliminate in-project training–test leakage, the largest external-validity concern for cross-project defect prediction (Zimmermann et al., 2009; Tantithamthavorn et al., 2017), we adopt a strict leave-one-project-out protocol. For each of the 33 projects P in the union of Defects4J and BugsInPy, we train the predictor on the process-metric rows of the other 32 projects, train the learned fusion on the (event, file) rows of the other 32 projects, and score the predictor, the rule fusion and the learned fusion on every event of P . No row from P contributes to the parameters of any classifier used to score P . The flat prior requires no training. Every metric we report is therefore an out-of-project estimate.

2.4 Metrics

For each event and each method we compute Precision@1 (1 if the top-ranked file is a real defect), Precision@3 (fraction of the top three that are real defects, capped near 1/3 for the 1,008 events with a single fix file), Recall@5 (fraction of the event’s real defects in the top five), mean reciprocal rank (MRR, $1/r^*$ with r^* the rank of the first real defect) and simulated triage seconds (a linear scan at 3s per file until the first real defect). Per-event values are averaged across events. Expected calibration error (ECE) is the 10-bin version of Guo et al. (Guo et al., 2017), computed over all (event, file) rows; it is the primary outcome for RQ1. We report every metric pooled and separately for Defects4J and BugsInPy, because the two halves differ.

Uncertainty is quantified by a paired, event-level non-parametric bootstrap: for each comparison against the predictor we resample the 1,267 events with replacement 10,000 times (seed 42) and report the mean paired difference in Precision@1 or MRR with its 2.5/97.5 percentile interval. Per-project wins, ties (within ± 1 percentage point) and losses are counted on point estimates, and a per-project paired bootstrap (2,000 resamples) marks a difference as significant when its 95% interval excludes zero.

2.5 Data sources

Defects4J v2.0.1 was obtained from <https://github.com/rjust/defects4j> (tag v2.0.1); each bug’s pre-fix and fix revisions are the `revision.id.buggy` and `revision.id.fixed` columns of the project’s `active-bugs.csv`, and the working copy produced by `defects4j checkout` carries the project’s git history, in which those revisions are resolved. BugsInPy was obtained from <https://github.com/soarsmu/BugsInPy> (default branch, snapshot of 2 October 2026); each bug’s `bug.info` supplies `buggy_commit_id`, `fixed_commit_id` and `github_url`, and the project repositories were cloned from those URLs. Both corpora are used under their respective licences, and neither is redistributed: the artifact contains our derived event table, the build log, and the scripts that regenerate the table from the two sources.

2.6 Corpus construction

The event table is built by one script, `build_real_events_v4.py`, in the following steps, which replace the construction used in an earlier version of this work (T12 in Section 4.6).

1. For every bug in the 17 Defects4J `active-bugs.csv` files (854 entries) and every bug in the 17 BugsInPy projects (501 entries), read the pre-fix and fix revisions.
2. Check out the pre-fix revision of the project repository. For Defects4J this is `defects4j checkout` of the buggy version, in whose git history `revision.id.buggy` is then used as the reference revision; for BugsInPy it is a checkout of `buggy_commit_id` in a clone of `github_url`. Every feature and every distractor pool is computed at the original pre-fix revision, so no commit at or after the fix contributes to any feature.
3. Take the fix files as the source files (`.java` or `.py`) whose path does not contain “test” in the diff between the pre-fix and fix revisions, capped at five. A bug whose fix touches only tests, resources or build files is skipped.
4. Draw distractors from the pool of source files of the same language, with no “test” in their path, in the same directories as the fix files at the pre-fix revision, excluding the fix files themselves. The number of distractors is drawn uniformly from the range that keeps the candidate set between 3 and 8 files, with a per-bug seed derived from the MD5 of the bug identifier so that the draw is reproducible, and the pool is shuffled with the same seed before the first n files are taken. A bug with no sibling source file is skipped, because a candidate set of fix files alone has nothing to rank.
5. Compute the seven process-metric features for every candidate from `git log -follow` of the file in the ancestry of the pre-fix revision, identically for both corpora.
6. Set the event-level columns: the five change-topology flags from the fix-file paths (for example `validator_changed` if a path contains “validate”, “validation” or “check”), and the exception type, HTTP status, locator-heal flag and flake rate to the single value each takes in these corpora (`AssertionError`, 0, 0 and 0.0), since recovering them would require re-executing every failing test.
7. Write one row per (event, file) with the binary label `real_defect`, and a build log with the status and skip reason of every bug.

Of the 1,355 bugs, 1,267 yield an event: 774 from Defects4J (16 projects) and 493 from BugsInPy (17 projects). Eighty-eight were skipped: all 26 Chart bugs (23 checkout failures and 3 pre-fix revisions that could not be resolved in the checkout’s history, which is why Chart is the one Defects4J project not covered), 37 bugs with no sibling source file (31 Defects4J, 6 BugsInPy), and 25 bugs whose fix touched no non-test source file (23 Defects4J, of which 14 in JacksonDatabind, and 2 BugsInPy). The corpus contains 6,716 candidate rows (4,178 Java, 2,538 Python), 5.30 files per event on average (between 2 and 8), 1.34 fix files per event, and a row-level defect fraction of 0.253. Every event carries an author-verified ground-truth label at the file level, inherited from Defects4J or BugsInPy without relabelling. Fix files are, as expected, hotter than their siblings: the median fix file has 36 commits and a churn of 1,475 lines against 17 commits and 480 lines for a distractor, with a commit frequency of 0.29 against 0.16 per week. That is the signal the predictor has; it is also why the task is not trivial, since the distributions overlap heavily.

2.7 Computing infrastructure

The corpus was built and the experiments run on a single Windows 11 laptop (Intel Core i7-12700H, 14 cores, 40 GB RAM, no discrete GPU) under WSL2 (Ubuntu 22.04), with OpenJDK 11 for the Defects4J framework, Python 3.10 (WSL) and 3.11 (analysis), pandas 2.x, scikit-learn 1.4 or later, numpy and pyarrow. Building the corpus took about three hours of wall-clock time,

dominated by repository checkouts and the first clone of the larger BugsInPy projects (pandas, keras, ansible, spacy); the leave-one-project-out experiment, the sensitivity analysis and the figures take a few minutes. The LLM baselines were run through the vendors’ HTTP APIs with keys held in environment variables; the run logs (one JSON per event, including the prompt and the raw response) are in the artifact. No GPU was used.

2.8 Use of generative AI in this study

Generative AI enters the study in two distinct ways. As study subjects, Claude Sonnet 4.6 and GPT-4o-mini are the two LLM attribution baselines of Section 2.2; every prompt and every raw response is logged in the artifact. As a tool, the author used Claude (Anthropic, via the Claude desktop application, July–October 2026) to help write and review the harness and analysis scripts, to check the consistency of the reported numbers against the result files, and to copy-edit the manuscript. All code was run, and all numbers were verified, by the author; the author takes full responsibility for the content.

3 Results

Table 1 reports all six methods on all 1,267 events, pooled and per corpus, and is referenced throughout. Figure 1 shows Precision@1 and ECE by method and corpus.

Table 1: Attribution-method comparison under leave-one-project-out on 1,267 real events (6,716 candidate rows, 33 projects), pooled and per corpus. ECE is 10-bin expected calibration error (lower is better); p@3 is capped near 1/3 for the 1,008 single-fix-file events; triage is a linear-scan cost at 3s per file. The rule fusion’s ranking metrics equal the predictor’s because its offset is constant within every event.

Corpus	Method	ECE	p@1	p@3	r@5	MRR	Triage (s)
Pooled (1,267)	Cross-project predictor	0.025	0.541	0.357	0.957	0.718	5.70
	Rule fusion	0.091	0.541	0.357	0.957	0.718	5.70
	Learned fusion	0.082	0.496	0.344	0.944	0.686	6.13
	Flat prior	0.747	0.243	0.264	0.882	0.490	8.85
	Claude Sonnet 4.6	0.065	0.532	0.356	0.964	0.709	5.85
	GPT-4o-mini	0.065	0.461	0.339	0.934	0.662	6.45
Defects4J (774)	Cross-project predictor	0.023	0.583	0.378	0.955	0.744	5.45
	Rule fusion	0.065	0.583	0.378	0.955	0.744	5.45
	Learned fusion	0.062	0.541	0.370	0.943	0.717	5.78
	Flat prior	0.731	0.258	0.275	0.869	0.499	8.84
	Claude Sonnet 4.6	0.069	0.589	0.373	0.956	0.743	5.60
	GPT-4o-mini	0.083	0.479	0.364	0.931	0.677	6.25
BugsInPy (493)	Cross-project predictor	0.065	0.475	0.323	0.960	0.679	6.09
	Rule fusion	0.135	0.475	0.323	0.960	0.679	6.09
	Learned fusion	0.114	0.424	0.303	0.947	0.637	6.70
	Flat prior	0.773	0.219	0.247	0.904	0.477	8.87
	Claude Sonnet 4.6	0.058	0.442	0.329	0.977	0.656	6.24
	GPT-4o-mini	0.045	0.432	0.300	0.940	0.638	6.75

3.1 RQ1 (trust): ranking and calibration on unseen projects

On ranking, the predictor places a fix file first in 54.1% of events, against 24.3% for the flat prior, a paired difference of 29.8 points (95% bootstrap interval 26.4 to 33.2). Its MRR is 0.718

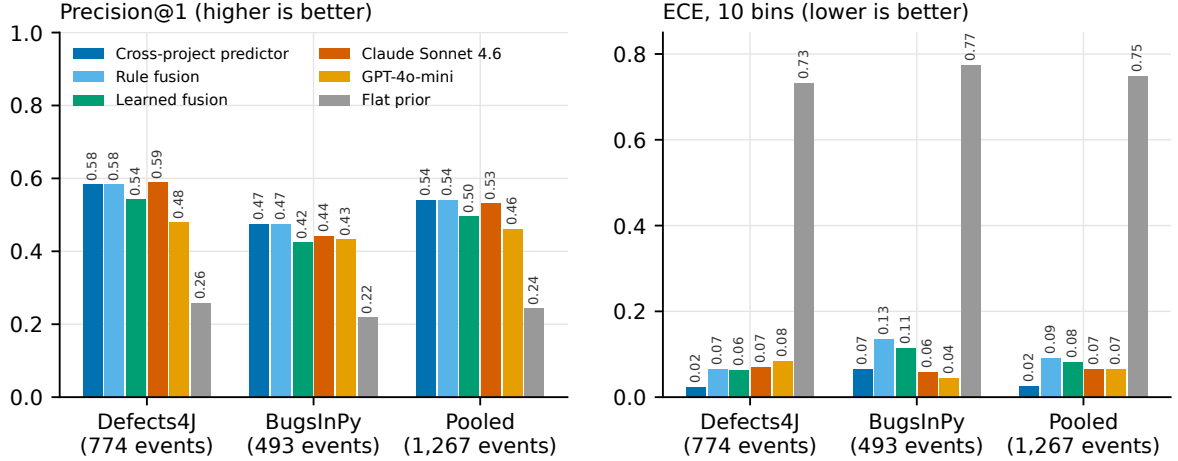


Figure 1: Precision@1 (left) and 10-bin expected calibration error (right) by method, for Defects4J, BugsInPy and the pooled corpus, under leave-one-project-out. The rule fusion’s Precision@1 is identical to the predictor’s by construction (Section 2.1).

and its Recall@5 0.957, so the fix file is almost always in the first five of a set that averages 5.3 files; the useful information is in the top of the list. The Java half is easier than the Python half (Precision@1 0.583 against 0.475), and we return to the per-project spread in Section 3.5.

On calibration, the predictor’s pooled ECE is 0.025 (0.023 on Defects4J, 0.065 on BugsInPy). Figure 2 decomposes it. Up to a predicted probability of 0.6 the predictor tracks the diagonal closely: the bins centred at 0.08, 0.14, 0.25, 0.35, 0.45 and 0.55 contain 1,078, 2,122, 1,181, 894, 623 and 496 rows and have observed defect fractions of 0.100, 0.136, 0.273, 0.315, 0.443 and 0.502. Above 0.6 the predictor is over-confident, and there is little data: the bins centred at 0.64, 0.74 and 0.83 contain 235, 74 and 13 rows with observed fractions of 0.532, 0.541 and 0.692. Only 4.8% of rows receive a probability above 0.6 and 1.3% above 0.7. Expressed as thresholds, the files scored at or above 0.5 are defective 51.7% of the time (62.6% on Java, 41.2% on Python); at or above 0.6, 54.0% (65.8%, 42.2%); at or above 0.7, 56.3% (73.5% on 49 Java rows, 34.2% on 38 Python rows). The answer to RQ1 is therefore qualified. A team can read the predictor’s probability at face value for the bulk of its range, which is where almost all files fall, and can act on a 0.5 threshold knowing that about half of what it selects is defective on an unseen project. It should not read a 0.7 as a 70% chance: on these corpora the high tail is thin and over-confident, and on Python in particular the files the predictor is most sure about are defective only a third of the time.

3.2 RQ2 (augment): the available failure signals cannot help, and fusion costs

The rule fusion’s ranking metrics are identical to the predictor’s on every event, as Section 2.1 explains: the only non-constant inputs to the rule are the change-topology flags, which are derived from the fix-file paths and so are the same for every candidate of an event. In 99.5% of events `app_code_changed` is set, `validator_changed` is set in 1.6%, `config_changed` in 0.6%, and the other two never. The rule therefore adds 0.08 to nearly every probability in the corpus, and the only thing it changes is the calibration, which worsens from 0.025 to 0.091 pooled (0.023 to 0.065 on Java, 0.065 to 0.135 on Python). That is the whole of the rule’s effect: a constant shift of a calibrated score away from the observed rate.

The learned fusion, which receives the same constant columns plus the predictor’s score and

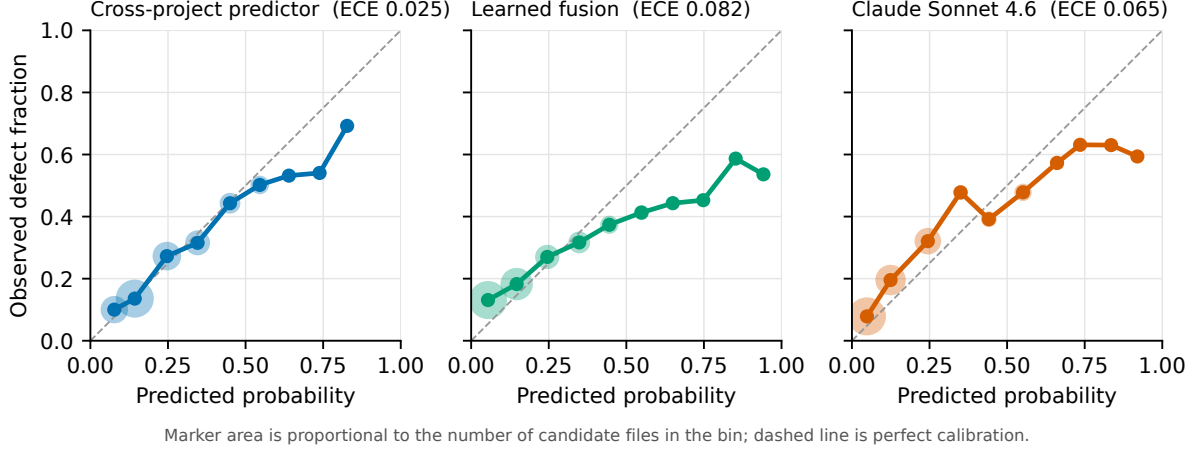


Figure 2: Reliability diagrams on the pooled corpus for the cross-project predictor, the learned fusion and Claude Sonnet 4.6. Marker area is proportional to the number of candidate rows in the bin; the dashed diagonal is perfect calibration. The predictor is calibrated up to 0.6 and over-confident above it, where fewer than 5% of rows fall.

the seven process metrics, is worse on both axes. Its Precision@1 is 0.496, a paired difference of -4.5 points against the predictor (95% interval -6.5 to -2.5 ; -4.1 points on Java and -5.1 on Python, both intervals excluding zero), its MRR is lower by 0.033 (-0.044 to -0.022), and its ECE is 0.082. Per project it loses to the predictor on 18 of 33, ties on 11 and wins on 4, with 4 of the differences significant at the per-project level (Table 3). The reliability diagram (Figure 2, middle) shows where the calibration goes: the learned fusion is under-confident at the bottom of the range and over-confident from about 0.35 upwards. Re-fitting a higher-capacity model on top of a calibrated score, with no new within-event information, has produced a less accurate ranking and a less trustworthy probability. The answer to RQ2 is no, and the mechanism is not subtle: the failure-time signals that can be recovered from these corpora without re-executing tests carry no information that distinguishes one candidate from another, so there is nothing for fusion to add, and both fusion designs pay for trying.

3.3 RQ3 (replace): a frontier LLM matches the predictor only where it knows the benchmark

Pooled, Claude Sonnet 4.6 is statistically indistinguishable from the predictor on ranking: Precision@1 0.532 against 0.541, a paired difference of -0.9 points with a 95% interval of -4.2 to $+2.5$, and an MRR difference of -0.009 (-0.030 to $+0.011$). Its Recall@5 of 0.964 is the highest of any method. GPT-4o-mini is clearly behind: Precision@1 0.461, a difference of -8.0 points (-11.0 to -5.0). On calibration both models are worse than the predictor, with pooled ECE 0.065 for each, 2.6 times the predictor’s 0.025; on the Python half, however, both models (0.058 and 0.045) are better calibrated than the predictor (0.065), and Claude’s reliability curve (Figure 2, right) is under-confident below 0.4 and over-confident above 0.5.

The pooled parity does not survive a look at where it comes from. Figure 3 plots per-project Precision@1 for the predictor and Claude Sonnet 4.6. Across the 33 projects Claude wins on 9, ties on 5 and loses on 19, and the three projects on which it wins by the largest margins are Lang, Math and Time, three of the Defects4J v1.2 projects on which the LLM program-repair literature has evaluated since its beginning (Xia et al., 2023). On those three projects (185 events) Claude ranks the fix file first in 89.7% of events against the predictor’s 51.4%, a paired difference of $+38.4$ points (95% interval $+29.2$ to $+47.6$). On the remaining 1,082 events,

the other 13 Defects4J projects and all of BugsInPy, Claude’s Precision@1 is 0.470 against the predictor’s 0.545, a difference of -7.6 points (-11.1 to -4.2). GPT-4o-mini shows no such bump: on Lang, Math and Time it is at 0.416, below the predictor, and on the other 1,082 events at 0.469, a difference of -7.7 points (-10.9 to -4.4), which is the same deficit Claude shows outside the three projects. The predictor itself does not find those three projects unusually easy (0.514 against 0.604 on the other 13 Defects4J projects). The split was chosen after seeing the per-project results, and we report it as a post-hoc observation rather than a pre-registered test; Closure and Mockito, which are also Defects4J v1 projects, show no such bump (Claude 0.560 and 0.595 against the predictor’s 0.675 and 0.703), so the pattern is specific to the three projects and not to the Defects4J v1 set as a whole.

The prompt shows the model the project name and the candidate file paths and nothing of the code. A model that attributes from the seven process-metric values should perform on Lang, Math and Time about as it performs elsewhere, as GPT-4o-mini does. A model that has seen these bugs during pre-training can recognise, from “Lang” and a path such as `src/main/java/org/apache/commons/lang3/math/NumberUtils.java`, which file the fix touched, and Claude’s 90% on exactly the three projects that Ramos et al. (Ramos et al., 2024) and the program-repair literature single out is what recall would look like. We cannot observe the model’s training data, and we state the finding at the level the data supports: Claude Sonnet 4.6’s parity with the predictor is a property of three widely studied projects and not of the task, and on the 30 projects where recall is less likely it trails the predictor by the same margin as the weaker model. The answer to RQ3 is that a prompted frontier model does not replace the predictor from the same information, and that an evaluation on Defects4J which does not separate Lang, Math and Time will overstate what it can do.

3.4 Sensitivity to candidate-set size

The headline numbers are measured at one candidate-set size (5.3 files, row-level defect fraction 0.253). To probe how they depend on it we rescored the corpus at $k \in \{2, 3, 4\}$ candidates per event: every fix file is retained and $k - 1$ distractors are drawn by a deterministic per-event subsample seeded from the MD5 of the event identifier, so the subsets are reproducible and nested; events with fewer than $k - 1$ distractors are skipped at that k . The scores are the ones already emitted under leave-one-project-out, so this is a rescoring, not a retraining. Table 2 and Figure 4 report the outcome.

Table 2: ECE (10-bin) and Precision@1 by candidate-set size k . Fix files are always retained; $k - 1$ distractors are drawn deterministically per event; “all” is the headline protocol. Shrinking the set raises the row-level defect fraction away from the 0.253 at which the models were fitted.

Method	$k = 2$		$k = 3$		$k = 4$		all	
	ECE	p@1	ECE	p@1	ECE	p@1	ECE	p@1
Cross-project predictor	0.276	0.792	0.113	0.654	0.030	0.569	0.025	0.541
Rule fusion	0.196	0.792	0.036	0.654	0.059	0.569	0.091	0.541
Learned fusion	0.267	0.765	0.133	0.608	0.083	0.524	0.082	0.496
Claude Sonnet 4.6	0.272	0.765	0.133	0.630	0.087	0.549	0.065	0.532
GPT-4o-mini	0.341	0.723	0.177	0.567	0.109	0.479	0.065	0.461
Events	1,267		1,195		948		1,267	
Defect fraction	0.573		0.390		0.291		0.253	

Two things are stable and one is not. The learned fusion is below the predictor on both axes at every k , and Claude Sonnet 4.6 is below the predictor on Precision@1 at every k (by 2.7, 2.4, 2.0 and 0.9 points), with GPT-4o-mini further behind, so the ranking order of the methods does

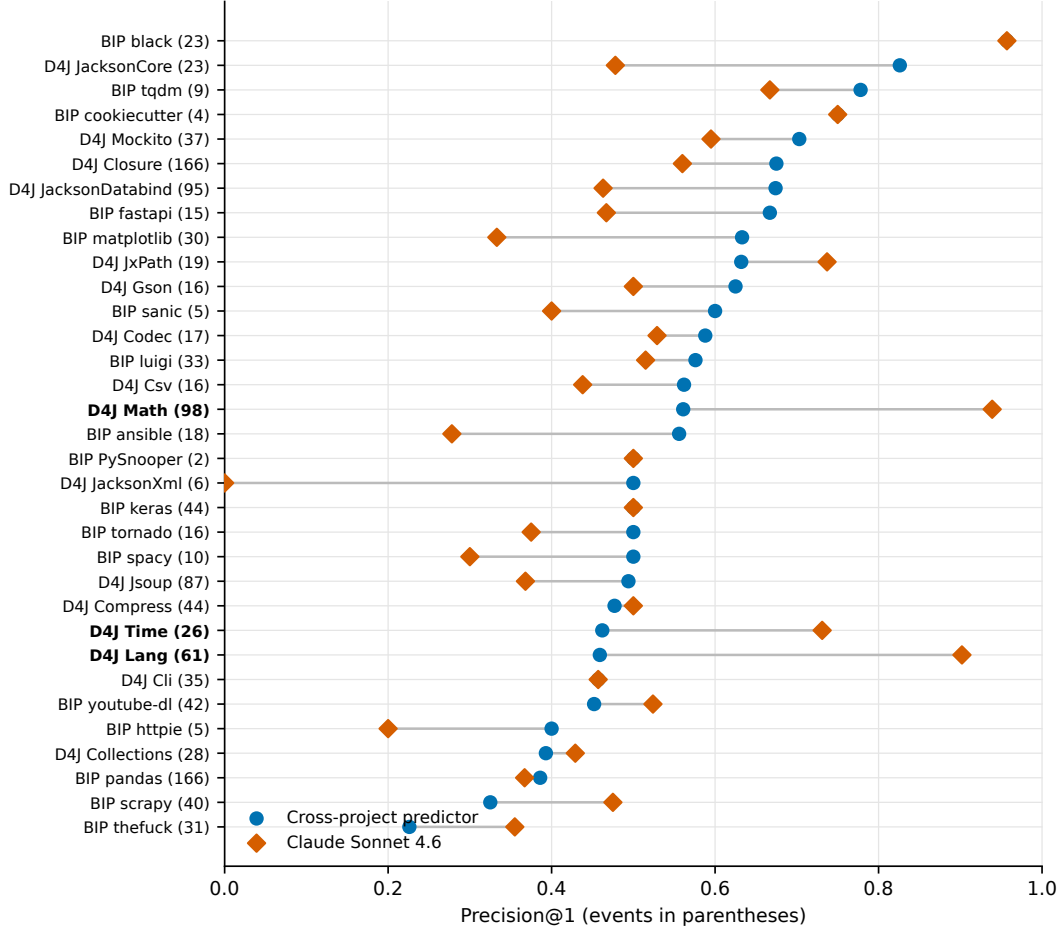


Figure 3: Per-project Precision@1 for the cross-project predictor (circles) and Claude Sonnet 4.6 (diamonds), 33 projects sorted by the predictor’s value, event counts in parentheses. The three bold labels are Lang, Math and Time, the Defects4J projects most used in LLM program-repair work; they account for the whole of Claude’s pooled parity with the predictor.

not depend on the candidate-set size. What is not stable is absolute calibration. As the set shrinks and the defect fraction rises from 0.253 to 0.573, every method’s ECE grows, because the probabilities were fitted at the lower rate; the predictor’s ECE goes from 0.025 to 0.276 at $k = 2$. The rule fusion’s constant +0.08 offset, which hurts at the designed rate, happens to help once the rate has moved up, and the rule is better calibrated than the predictor at $k = 3$ (0.036 against 0.113) and $k = 2$ (0.196 against 0.276). This is not evidence for the rule; it is a reminder that a calibration number is a property of the predictor at a base rate, and that a team whose candidate sets are smaller than ours should expect the predictor’s probabilities to run low.

3.5 Per-project decomposition and significance

Per-project Precision@1 ranges from 0.226 (thefuck, 31 events) to 0.957 (black, 23 events) for the predictor, and between-project variance is far larger than between-method variance for the trained methods. Table 3 counts, for every method, the projects on which it beats, ties or loses to the predictor, and how many of those differences a per-project paired bootstrap marks as significant. Most held-out projects contribute fewer than 50 events, so per-project significance is rare; the pooled paired intervals of the preceding sections are the primary evidence, and the

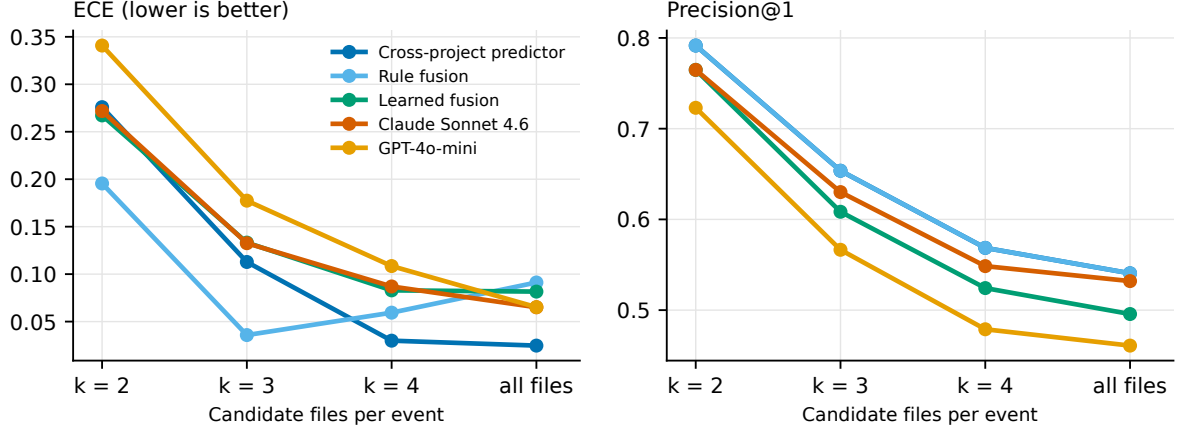


Figure 4: ECE (left) and Precision@1 (right) as the candidate set shrinks from the full corpus to two files per event. The defect fraction rises from 0.253 to 0.573 across the panels; calibration fitted at the lower rate deteriorates for every method, and the rule fusion’s constant +0.08 offset happens to compensate at $k = 2$ and $k = 3$.

per-project counts show that the pooled differences are broad patterns rather than single-project effects, with the one exception already discussed, Claude’s three Defects4J projects.

Table 3: Per-project comparison of each method against the cross-project predictor over the 33 held-out projects. Wins and losses are point-estimate Precision@1 differences larger than 1 percentage point; ties are within ± 1 point; significant counts the projects on which a per-project paired bootstrap (2,000 resamples) excludes zero at 95%.

Method vs predictor	Wins	Ties	Losses	Significant (either direction)
Rule fusion	0	33	0	0
Learned fusion	4	11	18	4
Flat prior	1	0	32	18
Claude Sonnet 4.6	9	5	19	8
GPT-4o-mini	4	4	25	4

3.6 Summary of empirical findings

Under leave-one-project-out on 1,267 real defects, the cross-project predictor ranks a fix file first in 54% of events (58% Java, 47% Python) against 24% for a flat prior, and its probabilities are calibrated up to 0.6 (ECE 0.025) and over-confident above it, where under 5% of files fall. The failure-time columns recoverable from these corpora are constant within an event, so the rule fusion reproduces the predictor’s ranking and triples its calibration error, and the learned fusion loses 4.5 points of Precision@1 and is three times worse calibrated. Claude Sonnet 4.6 matches the predictor’s pooled ranking, but only because of Lang, Math and Time, where it reaches 90%; on the other 30 projects it trails by 7.6 points, GPT-4o-mini trails by 8 points everywhere, and both are 2.6 times worse calibrated pooled. The method ordering is stable across candidate-set sizes; absolute calibration is not.

4 Discussion

4.1 What the calibration result does and does not license

An organisation that deploys a file-level predictor trained elsewhere can use its probability directly for ordering candidates and, within the range where almost all files fall, as a probability: a file scored 0.3 on a project the model has never seen is defective about three times in ten, and a 0.5 threshold selects files that are defective about half the time. That is a useful property that the fault-localization literature does not report (Yang et al., 2024; Kang et al., 2024; Rezaalipour and Furia, 2024), and it meets the practitioner concern documented at Atlassian (Hong et al., 2024) part of the way. What the result does not license is a high-confidence gate. The predictor’s probabilities above 0.6 are both rare and inflated, and on Python the files it is most sure about are wrong two times in three. A team that wants a 0.7 gate to mean 70% would need to recalibrate the tail on its own data or accept that the gate fires on a few dozen files a year with a precision nearer 55%. We report the per-language split because the pooled 0.025 hides a Python half at 0.065, and the direction is consistent: the predictor transfers to Java projects better than to Python ones in both ranking and calibration, which is the expected consequence of a corpus in which 62% of training rows are Java.

4.2 The fusion result is about what the corpora can supply

The fusion negative is not a finding about rules or gradient boosting; it is a finding about the inputs. Exception type, HTTP status, flake rate and locator-heal state cannot be recovered from Defects4J or BugsInPy without executing every failing test under instrumentation, and the change-topology flags, which can be derived from the fix diff, are by their nature event-level, so a per-file ranker cannot use them. Once that is said, the two fusion arms measure exactly one thing each. The rule measures what a constant offset does to a calibrated probability, and the answer is that it moves the probability off the observed rate by the size of the offset. The learned fusion measures what re-fitting a larger model on a calibrated score plus noise does, and the answer is a 4.5-point ranking loss and a calibration error three times larger, because the model has the capacity to fit per-fold idiosyncrasies in the training projects and no new signal to earn it back. The general caution survives the change of inputs: a fusion layer on top of a calibrated prior has an asymmetric profile, with an upside bounded by how much new within-event information it brings and a downside that is not, because its output is no longer the quantity the calibration was fitted on. Whether real failure-time signals, obtained by executing tests, would supply enough new information to pay for that downside is the question this study could not answer and that Section 5 leaves open.

4.3 What the LLM comparison shows

The parity design was meant to isolate reasoning over features from reading code, and for GPT-4o-mini it did: given the same seven numbers per file, the weaker model is 8 points behind the predictor everywhere, which measures what cross-project training extracts from process metrics that in-context reasoning does not. For Claude Sonnet 4.6 the design isolated something else. The model’s 90% on Lang, Math and Time, against 47% elsewhere, is not a pattern that reasoning over commit counts produces; the predictor, which has only the numbers, finds those projects slightly harder than the rest, and the weaker model finds them no easier. The pattern is what one would expect if the model recognises the bug from the project name and file paths, which the parity prompt does expose, and it is consistent with the memorisation that Ramos et al. (Ramos et al., 2024) measure on Defects4J directly. We present it as a finding about evaluation rather than as an accusation: any LLM result on Defects4J that pools Lang, Math

and Time with the rest should be read with the split in hand, and the file-level task here is sensitive enough to show it.

Outside those projects the picture is the one the parity design was built to reveal. The stronger model trails the predictor by 7.6 points on ranking while shortlisting slightly better (Recall@5 0.964 against 0.957), and its calibration is competitive on Python and worse on Java. There is a real signal in its confidences, since both models are ten times better calibrated than the flat prior, but a trained prior remains the better instrument for the number a gate acts on. The hybrid that these profiles suggest, a model proposing candidates from artefacts the predictor does not read and the predictor ordering and scoring them, is a hypothesis this data does not test, and any test of it on Defects4J would have to hold out the three projects.

4.4 Comparison to the fault-localization literature at large

Our Precision@1 of 0.54 at file level on a candidate set of about five look-alike files is not comparable to line-level or method-level numbers reported for Ochiai, DeepFL (Li et al., 2019), FLUCCS (Sohn and Yoo, 2017) or LLM-based localizers (Yang et al., 2024; Kang et al., 2024; Rezaalipour and Furia, 2024), which address a different granularity and a different input signal. The LLM comparison appropriate to our setting is the one we executed ourselves. The internal comparisons that matter are against the flat prior, which is today’s practice, and between the predictor and its augmentations, which is the adoption decision.

4.5 What a team should do

The three answers reduce to one rule with a caveat. If you adopt a file-level defect predictor trained on other projects, use its raw probability to order candidates, and trust it as a probability up to about 0.6; above that, expect it to run high until you have recalibrated the tail on your own projects, and expect the whole scale to run low if your candidate sets are smaller than five files. Do not wrap the predictor in a rule or a fusion model unless the signal you are adding varies between the files of one failure; the signals that are cheap to obtain do not, and adding them only moves the probability off its calibration. And do not expect a prompted language model to replace the predictor from the same inputs; where it appears to match, check whether the projects are ones it could have memorised, because on the projects where that is unlikely it ranks worse and calibrates worse than the trained model.

4.6 Threats to validity

We enumerate twelve threats and what we did, or did not do, about each.

T1: Chart excluded. All 26 Chart bugs were skipped: 23 because the Defects4J checkout of the pre-fix revision failed on our infrastructure and 3 because `revision.id.buggy` could not be resolved in the checkout’s history. The paper therefore covers 16 of the 17 Defects4J v2.0.1 projects. Chart is one of the Defects4J v1.2 projects on which LLM program-repair work has evaluated most (Xia et al., 2023), so its absence, if anything, removes events on which the contamination pattern of Section 3.3 would be expected to appear.

T2: Same-directory distractors. Candidates are the fix files plus source files from the same directories at the pre-fix revision. This is a deliberate choice between two alternatives. Drawing distractors from the whole repository would make the task trivial and the defect fraction tiny; drawing them from a coverage-instrumented run of the failing test, the set a mature CI system would surface, would be the better ground truth but requires re-executing 1,267 failing tests under language-specific coverage tools, which we did not

undertake. Same-directory siblings are files a developer would plausibly inspect and are of the same kind as the fix file, which makes the ranking task harder than a random draw and, we believe, closer to a coverage-derived set than the alternative. The residual bias is of unknown direction, and the sensitivity analysis shows that the absolute calibration numbers depend on the candidate-set size while the method ordering does not.

T3: Cross-language transfer. The leave-one-project-out folds mix Java and Python in every training set, and 62% of training rows are Java. The consistent Java advantage in Table 1 may partly reflect this. We did not evaluate a language-stratified variant.

T4: Feature extraction at the pre-fix revision. Process metrics are computed from `git log -follow` in the ancestry of the pre-fix revision, so no post-fix commit contributes. For Defects4J projects whose history was imported from Subversion, author and date fields can be coarser than in native git histories, which affects `unique_developers` and `file_age_days` uniformly across candidates of the same project.

T5: Leakage the protocol does not control. Leave-one-project-out controls in-project leakage but not shared-authorship leakage across projects or shared-framework leakage (JacksonCore, JacksonDatabind and JacksonXml share upstream design). We did not exclude cross-project author or framework overlap.

T6: Small projects. Several projects contribute fewer than 10 events (PySnooper 2, cookiecutter 4, httpie 5, sanic 5, JacksonXml 6, tqdm 9). Their per-project values carry wide intervals; we report them in Figure 3 and rely on the pooled paired bootstrap for inference.

T7: External validity beyond open source. Both corpora are open-source projects. Enterprise code may follow different defect distributions, review practices and test-suite compositions.

T8: Inheritance from an earlier synthetic study. The two fusion designs, their coefficients and hyperparameters come from an earlier study of ours on synthetic failure events in which the failure-side signals were generated and informative and the learned fusion appeared to help. On real defects those signals are not recoverable and both designs lose. We report the reversal rather than reframe it; nothing from the synthetic study is carried over as a conclusion, and the coefficients were held fixed rather than re-tuned, which is conservative for the predictor.

T9: Metric choice. Precision@ k , Recall@ k , MRR and 10-bin ECE reward top-of-list ranking and probability calibration. Task-oriented measures such as developer debugging time would require a user study. ECE with ten equal-width bins is sensitive to the thin high tail; the per-bin counts are reported so that readers can judge.

T10: Ground-truth completeness. Both corpora label one fix commit per bug, and we take its non-test source files as the real defects. If the actual defect spans a file the fix did not touch (Herzig et al., 2013), every method’s precision is understated uniformly.

T11: LLM-baseline construct validity and contamination. The LLM baselines observe engineered features rather than code, traces or diffs, so RQ3 bounds LLM-over-features, not LLM capability as a fault localizer. The prompt is single-shot at temperature 0 without few-shot examples or agentic iteration, each model was run once, and the results are a snapshot of specific model versions accessed in October 2026. Benchmark contamination is not a threat we can rule out; it is a finding we report, and the split in Section 3.3 is our attempt to bound it. The prompt exposes project names and file paths because the predictor’s inputs include file identity; a variant with anonymised paths would remove the recognition channel at the cost of removing path semantics from both methods, and we did not run it.

T12: Relation to the earlier version of this corpus. An earlier version of this study, archived as version 3.0.0 of the artifact, used a corpus whose Defects4J features were computed at a synthetic post-fix tag, whose BugsInPy features were proxies derived from fix-pool frequencies rather than git history, and which used two different distractor schemes. Those constructions leaked the label into the Java features and made the Python features label-adjacent, and the numbers reported from them (Precision@1 of 0.78, ECE of 0.021) are superseded by this paper. The corrected construction is the one described in Section 2.6; the artifact’s changelog records the differences, and version 3.0.0 remains available for comparison.

5 Conclusions

A team that adopts a defect predictor trained on other projects wants to know whether to trust it, whether to augment it and whether to replace it. On 1,267 real defects from Defects4J and BugsInPy across 33 projects, with every prediction made for a project excluded from training and every candidate set made of files that look alike, the answers are a qualified yes, no, and not from attribution. The predictor ranks a fix file first in 54% of events against 24% for a flat prior, and its probabilities can be read at face value up to about 0.6, which covers 95% of files; above that it is over-confident, especially on Python. The failure-time signals that two standard corpora can supply are constant within an event, so augmenting the predictor with them cannot improve the ranking and only costs calibration, by a factor of three for a rule and with a 4.5-point ranking loss as well for a learned fusion. A frontier language model given the predictor’s inputs matches it only on the three Defects4J projects that the program-repair literature has studied most, where it reaches 90%; on the other 30 projects it trails by 7.6 points and calibrates worse, and a weaker model trails by 8 points everywhere.

Limitations. The study has four limitations that bound its claims. First, the candidate sets are same-directory siblings rather than coverage-derived implicated sets, and the absolute calibration numbers depend on the candidate-set size, as the sensitivity analysis shows; the method ordering does not. Second, the failure-time signals the fusion arms were designed to consume are not recoverable from Defects4J or BugsInPy without re-executing tests, so the fusion result bounds what cheaply available signals can do, not what instrumented signals could. Third, the LLM comparison is a snapshot of two model versions under a single-shot feature-only prompt, and the contamination finding is inferred from the pattern of results rather than observed in training data. Fourth, Chart is excluded and the corpus is 61% Java by events, so the Java–Python differences are confounded with training-set composition. Two directions follow. Replacing sibling distractors with coverage-derived sets, and recovering real exception and flake signals by executing the failing tests, would test both the calibration and the fusion results on the inputs a CI system actually has. And any further LLM evaluation on Defects4J should report Lang, Math, Time and Chart separately from the rest, or anonymise project names and paths, before reading parity as capability.

Acknowledgements

The Defects4J and BugsInPy maintainers made this study possible by curating and maintaining these open-source real-defect corpora. Claude (Anthropic) was used under the author’s direction for coding assistance, number checking and copy-editing, as described in Section 2.8.

Data and code availability

The 1,267-event corpus (6,716 candidate-file rows) with its build log, the corpus-construction script, the four method implementations, the leave-one-project-out harness, the LLM prompts and raw per-event responses, the sensitivity, bootstrap and analysis scripts, and every figure and table in this paper are archived on Zenodo as version 4.0.0 of the artifact, 10.5281/zenodo.23111801 (concept DOI for all versions 10.5281/zenodo.20723929; version 3.0.0 is the superseded construction described in T12) and mirrored at github.com/javvadivijayprasad/DefectAnalysisResearch. Defects4J and BugsInPy themselves are available from the sources given in Section 2.5.

Funding

The author received no funding for this work. The LLM baselines used API access paid for by the author (US\$7.55 for Claude Sonnet 4.6; the GPT-4o-mini run incurred no charge).

Competing interests

The author declares no competing interests.

Declaration of generative AI use

Two large language models, Claude Sonnet 4.6 and GPT-4o-mini, are subjects of the experiment (Section 2.2); their prompts and raw responses are logged in the artifact. Claude (Anthropic, Claude desktop application, July–October 2026) was additionally used as a coding, number-checking and copy-editing assistant under the author’s direction (Section 2.8). The author takes full responsibility for the content.

References

- Rui Abreu, Peter Zoetewij, and Arjan J. C. van Gemund. Spectrum-based multiple fault localization. In *Proceedings of the 24th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 88–99. IEEE, 2009. doi: 10.1109/ASE.2009.25. DOI: 10.1109/ASE.2009.25.
- Christian Bird, Adrian Bachmann, Eirik Aune, John Duffy, Abraham Bernstein, Vladimir Filkov, and Premkumar Devanbu. Fair and balanced? bias in bug-fix datasets. In *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 121–130, 2009. doi: 10.1145/1595696.1595716. DOI: 10.1145/1595696.1595716.
- Moataz Chouchen, Ali Ouni, Gopi Krishnan Rajbahadur, Marouane Kessentini, and Mohamed Wiem Mkaouer. Less is more: Balancing models performance and complexity for software defects prediction. *Empirical Software Engineering*, 32(1):5, 2026. doi: 10.1007/s10664-026-10913-4. DOI: 10.1007/s10664-026-10913-4.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.

- Kim Herzig, Sascha Just, and Andreas Zeller. It's not a bug, it's a feature: How misclassification impacts bug prediction. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 392–401. IEEE, 2013. doi: 10.1109/ICSE.2013.6606585. DOI: 10.1109/ICSE.2013.6606585.
- Yang Hong, Chakkrit Tantithamthavorn, Jirat Pasuksmit, Patanamon Thongtanunam, Arik Friedman, Xing Zhao, and Anton Krasikov. Practitioners' challenges and perceptions of CI build failure predictions at Atlassian. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion '24)*, pages 370–381. ACM, 2024. doi: 10.1145/3663529.3663856. DOI: 10.1145/3663529.3663856.
- René Just, Darioush Jalali, and Michael D. Ernst. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA)*, pages 437–440. ACM, 2014. doi: 10.1145/2610384.2628055. DOI: 10.1145/2610384.2628055.
- Yasutaka Kamei, Emad Shihab, Bram Adams, Ahmed E. Hassan, Audris Mockus, Anand Sinha, and Naoyasu Ubayashi. A large-scale empirical study of just-in-time quality assurance. *IEEE Transactions on Software Engineering*, 39(6):757–773, 2013. doi: 10.1109/TSE.2012.70. DOI: 10.1109/TSE.2012.70.
- Sungmin Kang, Gabin An, and Shin Yoo. A quantitative and qualitative evaluation of LLM-Based explainable fault localization. *Proceedings of the ACM on Software Engineering (FSE)*, 1(FSE), 2024. doi: 10.1145/3660771. DOI: 10.1145/3660771.
- Xia Li, Wei Li, Yuqun Zhang, and Lingming Zhang. DeepFL: Integrating multiple fault diagnosis dimensions for deep fault localization. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 169–180. ACM, 2019. doi: 10.1145/3293882.3330574. DOI: 10.1145/3293882.3330574.
- Zhixing Li, Qiao Du, Hongyu Zhang, Yaohui Wang, and Xin Chen. An empirical study of data sampling techniques for just-in-time software defect prediction. *Automated Software Engineering*, 31(2):56, 2024. doi: 10.1007/s10515-024-00455-8. DOI: 10.1007/s10515-024-00455-8.
- Raimund Moser, Witold Pedrycz, and Giancarlo Succi. A comparative analysis of the efficiency of change metrics and static code attributes for defect prediction. In *Proceedings of the 30th International Conference on Software Engineering (ICSE)*, pages 181–190, 2008. doi: 10.1145/1368088.1368114. DOI: 10.1145/1368088.1368114.
- Nachiappan Nagappan and Thomas Ball. Use of relative code churn measures to predict system defect density. In *Proceedings of the 27th International Conference on Software Engineering (ICSE)*, pages 284–292, 2005. doi: 10.1109/ICSE.2005.1553571. DOI: 10.1109/ICSE.2005.1553571.
- Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 625–632. ACM, 2005. doi: 10.1145/1102351.1102430. DOI: 10.1145/1102351.1102430.
- Owain Parry, Gregory M. Kapfhammer, Michael Hilton, and Phil McMinn. Systemic flakiness: An empirical analysis of co-occurring flaky test failures. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE '25)*, pages 476–487. ACM, 2025. doi: 10.1145/3756681.3756945. DOI: 10.1145/3756681.3756945.

- Salomé Perez-Rosero, Robert Dyer, Samuel W. Flint, Shane McIntosh, and Witawas Srisa-
An. WIA-SZZ: Work item aware SZZ. *Empirical Software Engineering*, 30(3):75, 2025. doi:
10.1007/s10664-025-10616-2. DOI: 10.1007/s10664-025-10616-2.
- Foyzur Rahman and Premkumar Devanbu. How, and why, process metrics are better. In
Proceedings of the 35th International Conference on Software Engineering (ICSE), pages 432–
441, 2013. doi: 10.1109/ICSE.2013.6606589. DOI: 10.1109/ICSE.2013.6606589.
- Daniel Ramos, Claudia Mamede, Kush Jain, Paulo Canelas, Christopher Timperley, and
Claire Le Goues. Are large language models memorizing bug benchmarks? *arXiv preprint*
arXiv:2411.13323, 2024.
- Pooja Rani, Fernando Petrulio, and Alberto Bacchelli. On refining the SZZ algorithm with
bug discussion data. *Empirical Software Engineering*, 29(5):115, 2024. doi: 10.1007/
s10664-024-10511-2. DOI: 10.1007/s10664-024-10511-2.
- Mohammad Rezaalipour and Carlo A. Furia. An empirical study of fault localization in Python
programs. *Empirical Software Engineering*, 29(4):92, 2024. doi: 10.1007/s10664-024-10475-3.
DOI: 10.1007/s10664-024-10475-3.
- Giovanni Rosa, Luca Pascarella, Simone Scalabrino, Rosalia Tufano, Gabriele Bavota, Michele
Lanza, and Rocco Oliveto. Evaluating SZZ implementations through a developer-informed or-
acle. *Proceedings of the 43rd International Conference on Software Engineering (ICSE)*, pages
436–447, 2021. doi: 10.1109/ICSE43902.2021.00049. DOI: 10.1109/ICSE43902.2021.00049.
- Xhulja Shahini, Andreas Metzger, and Klaus Pohl. An empirical study on just-in-time conformal
defect prediction. In *Proceedings of the 21st International Conference on Mining Software*
Repositories (MSR ’24), pages 88–99. ACM, 2024. doi: 10.1145/3643991.3644928. DOI:
10.1145/3643991.3644928.
- Jacek Śliwerski, Thomas Zimmermann, and Andreas Zeller. When do changes induce fixes?
In *Proceedings of the 2005 International Workshop on Mining Software Repositories (MSR)*,
pages 1–5, 2005. doi: 10.1145/1083142.1083147. DOI: 10.1145/1083142.1083147.
- Jeongju Sohn and Shin Yoo. FLUCCS: Using code and change metrics to improve fault lo-
calization. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software*
Testing and Analysis (ISSTA), pages 273–283. ACM, 2017. doi: 10.1145/3092703.3092717.
DOI: 10.1145/3092703.3092717.
- Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, and Kenichi Matsumoto. An
empirical comparison of model validation techniques for defect prediction models. *IEEE*
Transactions on Software Engineering, 43(1):1–18, 2017. doi: 10.1109/TSE.2016.2584050.
DOI: 10.1109/TSE.2016.2584050.
- Bogdan Vasilescu, Yue Yu, Huaimin Wang, Premkumar Devanbu, and Vladimir Filkov. Quality
and productivity outcomes relating to continuous integration in GitHub. In *Proceedings of the*
10th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT
Symposium on the Foundations of Software Engineering (ESEC/FSE), pages 805–816. ACM,
2015. doi: 10.1145/2786805.2786850. DOI: 10.1145/2786805.2786850.
- Ratnadira Widayarsi, Sheng Qin Sim, Camellia Lok, Haodi Qi, Jack Phan, Qijin Tay, Constance
Tan, Fiona Wee, Jodie Ethelda Tan, Yuheng Yieh, Brian Goh, Ferdian Thung, Hong Jin
Kang, Thong Hoang, David Lo, and Eng Lieh Ouh. BugsInPy: A database of existing bugs in
Python programs to enable controlled testing and debugging studies. In *Proceedings of the 28th*
ACM Joint European Software Engineering Conference and Symposium on the Foundations

of *Software Engineering (ESEC/FSE)*, pages 1556–1560. ACM, 2020. doi: 10.1145/3368089.3417943. DOI: 10.1145/3368089.3417943.

Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 1482–1494, 2023. doi: 10.1109/ICSE48619.2023.00129.

Aidan Z. H. Yang, Claire Le Goues, Ruben Martins, and Vincent J. Hellendoorn. Large language models for test-free fault localization. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*, pages 17:1–17:12. ACM, 2024. doi: 10.1145/3597503.3623342. DOI: 10.1145/3597503.3623342.

Xin Zhou, DongGyun Han, and David Lo. Bridging expert knowledge with deep learning techniques for just-in-time defect prediction. *Empirical Software Engineering*, 30(2):37, 2025. doi: 10.1007/s10664-024-10591-0. DOI: 10.1007/s10664-024-10591-0.

Thomas Zimmermann, Nachiappan Nagappan, Harald Gall, Emanuel Giger, and Brendan Murphy. Cross-project defect prediction: A large scale experiment on data vs. domain vs. process. In *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 91–100, 2009. doi: 10.1145/1595696.1595713. DOI: 10.1145/1595696.1595713.